Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 17
All-Pairs Shortest Paths (Part 2)
Πανζευκτικές Ελαφρύτατες Διαδρομές (Μέρος 2)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, E. D. Demaine, and M. Christodoulakis

1

All-Pairs Shortest Paths
(Πανζευκτικές Ελαφρύτατες Διαδρομές)

- Input: Directed graph $G = (V, E)$, where $V = \{1, 2, \ldots, n\}$, with edge-weight function $w: \mathrm{E} \to \mathbb{R}$
- Output: $n \times n$ matrix $D = (d_{ij})$ of shortest-path lengths $\delta(i, j)$ for all $i, j \in V$
- Most algorithms we'll see use an **adjacency-matrix** (**πίνακας γειτνίασης**) $W = (w_{ij})$
  - Johnson's uses adjacency lists for sparse graphs

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ \text{weight of edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

2

Floyd-Warshall Algorithm

- Also dynamic programming, but faster!
- For path $p = \langle v_1, v_2, \ldots, v_l \rangle$, an **intermediate vertex** (**ενδιάμεση κορυφή**) is any vertex of p other than v_1 or v_l
- Define:

  $d_{ij}^{(k)}$ = weight of a shortest path from i to j with intermediate vertices belonging to the set $\{1,2,\ldots,k\}$



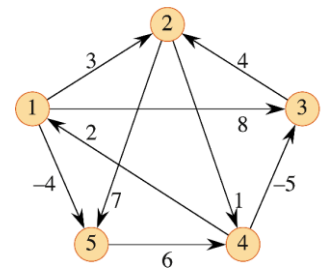
- Thus, $\delta(i,j) = d_{ij}^{(n)}$ (*Why?*)

Floyd-Warshall Algorithm

- Define:

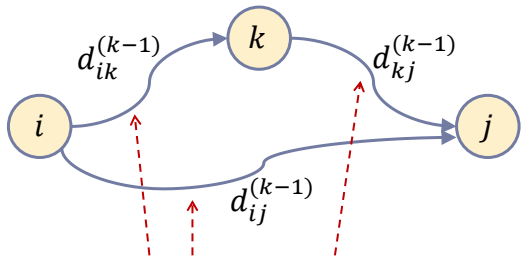
  $d_{ij}^{(k)}$ = weight of a shortest path from i to j with intermediate vertices belonging to the set $\{1,2,\ldots,k\}$

- Examples:
  - $d_{42}^{(1)} = 5$
    The shortest path from node 4 to node 2 with intermediate vertices up to node 1 is $4 \to 1 \to 2$
  - $d_{42}^{(4)} = -1$
    The shortest path from node 4 to node 2 with intermediate vertices up to node 4 is $4 \to 3 \to 2$



Floyd-Warshall Recurrence

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min\left\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right\} & \text{if } k \geq 1 \end{cases}$$



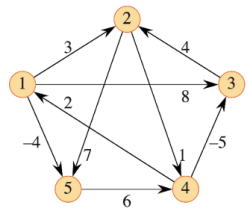
all intermediate vertices in $\{1,2,\ldots,k-1\}$

Pseudocode for Floyd-Warshall

```
FLOYD-WARSHALL(W, n) {
    D ← W
    for k ← 1 to n
        for i ← 1 to n
            for j ← 1 to n
                if d_ij > d_ik + d_kj
                    d_ij ← d_ik + d_kj        } relaxation
    return D
}
```

- Efficient in practice (hidden constants are small)
- What is the running time? $\Theta(n^3)$

Example of All-Pairs Shortest Path



$$d_{ij} = \min\{d_{ij}, d_{ik} + d_{kj}\}$$

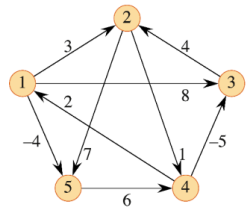
$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

- Intuition: To find (i,j) entry in $D^{(1)}$, compare (i,j) with $(i,1) + (1,j)$ in $D^{(0)}$
- Example:

$$\begin{aligned} d_{42}^{(1)} &= \min\left\{d_{42}^{(0)}, d_{41}^{(0)} + d_{12}^{(0)}\right\} \\ &= \min\{\infty, 2 + 3\} \\ &= 5 \end{aligned}$$

Example of All-Pairs Shortest Path



$$d_{ij} = \min\{d_{ij}, d_{ik} + d_{kj}\}$$

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Johnson's Algorithm

- Suitable for sparse graphs

1. If there are no negative-weight edges, then run Dijkstra once from each vertex
   - Running time $O(V^2 \lg V + VE)$ using Fibonacci heap
2. If there are negative-weight edges (but no negative-weight cycles), **reweight** the graph such that all the edge weights become non-negative, and run Dijkstra as above
   - Reweighting must preserve shortest paths!

Graph Reweighting

- **Theorem:** Given a function $h: \mathrm{V} \to \mathbb{R}$, reweight each edge $(u, v) \in E$ by $w_h(u, v) = w(u, v) + h(u) - h(v)$, where $h(u)$ is a real number associated with u. Then, for any two vertices, all paths between them are reweighted by the same amount

- **Proof:** Let $p = v_o \to v_1 \to \cdots \to v_k$ be a path in G. Then

$$\begin{aligned} w_h(p) &= \sum_{i=1}^{k} w_h(v_{i-1}, v_i) \\ &= \sum_{i=1}^{k} \big(w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)\big) \\ &= \sum_{i=1}^{k} w(v_{i-1}, v_i) + h(v_0) - h(v_k) \\ &= w(p) + \boxed{h(v_0) - h(v_k)} \end{aligned}$$

Same amount

Shortest Paths in Reweighted Graphs

- **Corollary 1:** $\delta_h(u,v) = \delta(u,v) + h(u) - h(v)$
  - $w_h(p) = w(p) + h(u) - h(v)$ for path p from u to v
  - $h(u), h(v)$ do not depend on the path from u to v
  - Hence, if one path $u \rightsquigarrow v$ is shorter than another with w, it is also shorter with w_h
- **Corollary 2:** G has a negative-weight cycle if and only if G_h has a negative-weight cycle
  - Let cycle $c = v_o \rightarrow v_1 \rightarrow \cdots \rightarrow v_k$
  - Then, $w_h(c) = w(c) + h(v_0) - h(v_k) = w(c)$
    - *Why?* $v_o = v_k \Rightarrow h(v_o) = h(v_k)$

Producing Nonnegative Weights

- Create a new graph $G' = (V', E')$
  - $V' = V \cup \{s\}$, where s is a new vertex
  - $E' = E \cup \{(s,v): v \in V\}$
  - $w(s,v) = 0$ for all $v \in V$
- Note: shortest paths $\delta(u,v)$ for $u \neq s$ and $v \neq s$ do not change (*Why?*)
- Define $h(v) = \delta(s,v)$
  - Note: $\delta(s,v) \leq \delta(s,u) + w(u,v)$ (triangle inequality)
    $\Rightarrow h(v) \leq h(u) + w(u,v)$
- Then, the new weight function w_h satisfies
  - $w_h(u,v) = w(u,v) + h(u) - h(v) \geq 0$

Johnson's algorithm

Johnson(G, w)

1 compute G', where $G'.V = G.V \cup \{s\}$,
  $G'.E = G.E \cup \{(s, v): v \in G.V\}$, and
  $w(s, v) = 0$ for all $v \in G.V$
2 **if** BellmanFord$(G', w, s) ==$ FALSE
3 print "the input graph contains a negative-weight cycle"
4 **else for** each vertex $v \in G'.V$
5 set $h(v)$ to the value of $\delta(s, v)$
  computed by the Bellman-Ford algorithm
6 **for** each edge $(u, v) \in G'.E$
7 $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$
8 let $D = (d_{uv})$ be a new $n \times n$ matrix
9 **for** each vertex $u \in G.V$
10 run Dijkstra$(G, \hat{w}, u)$ to compute $\hat{\delta}(u, v)$ for all $v \in G.V$
11 **for** each vertex $v \in G.V$
12 $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
13 **return** D

Running Times?

? $\Theta(V + E)$

? $\Theta(VE)$

? $\Theta(V)$

? $\Theta(E)$

? $\Theta(V^2 \lg V + VE)$

? $\Theta(V^2)$

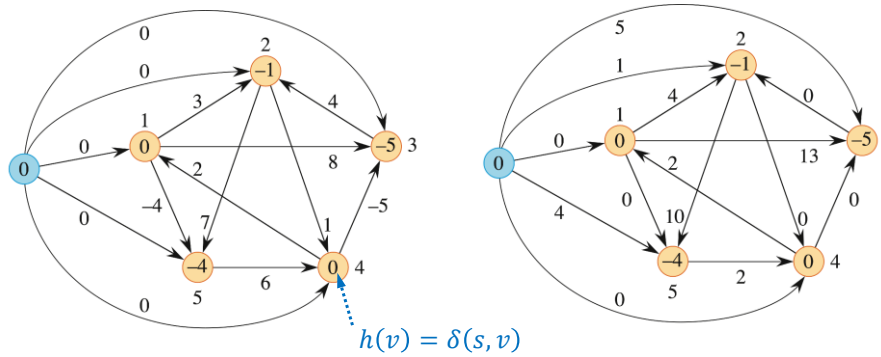
Sparse graphs: $\Theta(V^2 \lg V)$

Johnson's Algorithm: Example

G' before reweighting

G' after reweighting

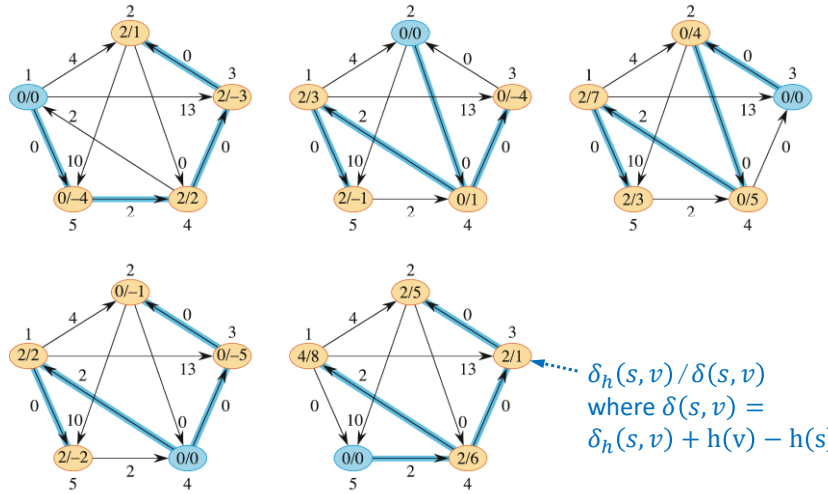
$w_h(u, v) = w(u, v) + h(u) - h(v)$



$h(v) = \delta(s, v)$

$h(1) = 0, h(2) = -1, h(3) = -5, h(4) = 0, h(5) = -4$

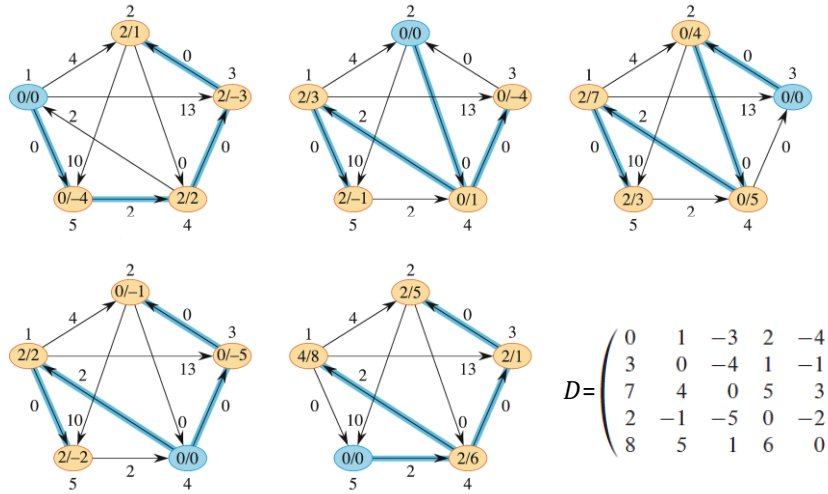
Johnson's Algorithm: Example



$\delta_h(s,v)/\delta(s,v)$ where $\delta(s,v) = \delta_h(s,v) + \mathrm{h(v)} - \mathrm{h(s)}$

$h(1) = 0, h(2) = -1, h(3) = -5, h(4) = 0, h(5) = -4$

Johnson's Algorithm: Example



$$D = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

$h(1) = 0, h(2) = -1, h(3) = -5, h(4) = 0, h(5) = -4$